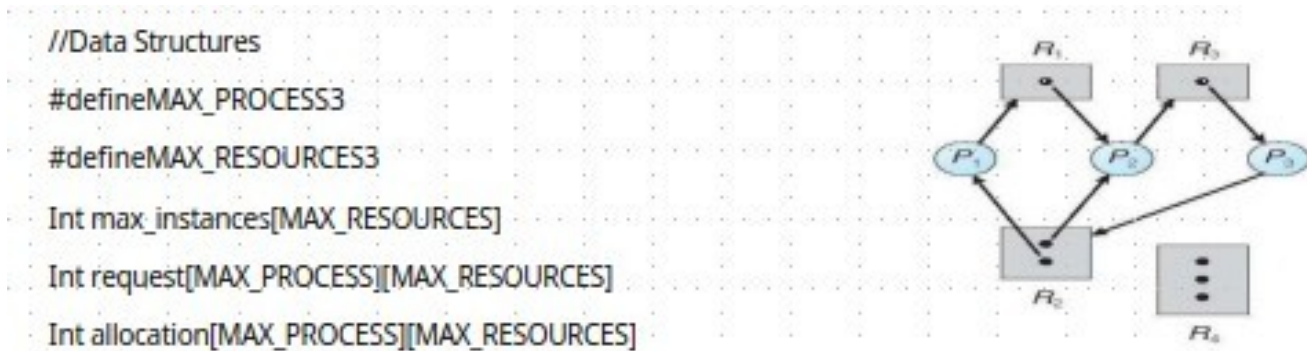


Lab#12

Implement the Banker's Algorithm to achieve deadlock avoidance in resource allocation systems, enhancing the reliability and efficiency of process management in operating systems

Task1:

Given the resource allocation graph below. Write the code to initialize the maximum instances of each resource using for loop.



Code:

```
#define RESOURCES 4

int main()
{
    int max_instances[RESOURCES];

    max_instances[0] = 1; // R1
    max_instances[1] = 2; // R2
    max_instances[2] = 1; // R3
    max_instances[3] = 3; // R4

    printf("Maximum instances of each resource:\n");
    for (int i = 0; i < RESOURCES; i++)
        printf("R%d = %d\n", i + 1, max_instances[i]);

    return 0;
}
```

Output:

```
muhammad-ahmad@latitude-7490:~/os/lab12$ ./Task1.out
Maximum instances of each resource:
R1 = 1
R2 = 2
R3 = 1
R4 = 3
```

Task 2: write the code to initialize the request matrix according to the RAG shown and print the matrix.

```
#include <stdio.h>

#define P 3
#define R 4

int main()
{
    int request[P][R] = {
        {1, 0, 0, 0}, // P1 → R1
        {0, 0, 1, 0}, // P2 → R3
    }
```

```

        {0, 1, 0, 0} // P3 → R2
    };

    printf("Request Matrix:\n");
    for (int i = 0; i < P; i++)
    {
        for (int j = 0; j < R; j++)
            printf("%d ", request[i][j]);
        printf("\n");
    }
    return 0;
}

```

Output:

```

muhammad-ahmad@latitude-7490:~/os/lab12$ ./Task2.out
Request Matrix:
1 0 0 0
0 0 1 0
0 1 0 0

```

Task 3:

Write the code to print the available resource(instances) vector.

Code:

```

#include <stdio.h>

#define R 4
int main()
{
    int max_instances[R] = {1, 2, 1, 3};
    int allocation_sum[R] = {1, 1, 1, 0};

    int available[R];

    for (int i = 0; i < R; i++)
        available[i] = max_instances[i] - allocation_sum[i];

    printf("Available Resource Vector:\n");
    for (int i = 0; i < R; i++)
        printf("R%d = %d\n", i + 1, available[i]);

    return 0;
}

```

Output:

```

muhammad-ahmad@latitude-7490:~/os/lab12$ ./Task3.out
Available Resource Vector:
R1 = 0
R2 = 1
R3 = 0
R4 = 3

```

Task 4:

Write the code to prompt the user to enter the maximum required instances of each resource by the each process. Also print the maximum required matrix.(Assume 3 processes and 3 resources). Make sure the maximum required instances by each process must not exceed maximum instances of each resource.

//Data Structure

required/involved Int

Max_required[3][3];

Int max_instances[3];

Code:

```
#include <stdio.h>

#define P 3
#define R 4

int main()
{
    int max_instances[R] = {1, 2, 1, 3};
    int max_required[P][R];

    for (int i = 0; i < P; i++)
    {
        for (int j = 0; j < R; j++)
        {
            do
            {
                printf("Enter max requirement of P%d for R%d: ", i + 1, j +
1);
                scanf("%d", &max_required[i][j]);
            } while (max_required[i][j] > max_instances[j]);
        }
    }

    printf("\nMaximum Required Matrix:\n");
    for (int i = 0; i < P; i++)
    {
        for (int j = 0; j < R; j++)
            printf("%d ", max_required[i][j]);
        printf("\n");
    }

    return 0;
}
```

Output:

```
muhammad-ahmad@latitude-7490:~/os/lab12$ ./Task4.out
Enter max requirement of P1 for R1: 0
Enter max requirement of P1 for R2: 1
Enter max requirement of P1 for R3: 0
Enter max requirement of P1 for R4: 2
Enter max requirement of P2 for R1: 0
Enter max requirement of P2 for R2: 0
Enter max requirement of P2 for R3: 0
Enter max requirement of P2 for R4: 3
Enter max requirement of P3 for R1: 0
Enter max requirement of P3 for R2: 1
Enter max requirement of P3 for R3: 0
Enter max requirement of P3 for R4: 2

Maximum Required Matrix:
0 1 0 2
0 0 0 3
0 1 0 2
```

Task 5:

Write the code to calculate the need matrix for the processes.(Assume 3 processes and 3 resources)

```
//Data Structure Int need[3][3];
```

```
Int Max_required[3][3]; Int allocated[3][3];
```

Code:

```
#include <stdio.h>
#define P 3
#define R 4

int main()
{
    int allocation[P][R] = {
        {0, 1, 0, 0},
        {1, 0, 0, 0},
        {0, 0, 1, 0}
    };

    int max_required[P][R] = {
        {1, 1, 0, 0},
        {1, 0, 1, 0},
        {0, 2, 1, 0}
    };

    int need[P][R];

    for (int i = 0; i < P; i++)
        for (int j = 0; j < R; j++)
            need[i][j] = max_required[i][j] - allocation[i][j];

    printf("Need Matrix:\n");
```

```

    for (int i = 0; i < P; i++)
    {
        for (int j = 0; j < R; j++)
            printf("%d ", need[i][j]);
        printf("\n");
    }

    return 0;
}

```

Output:

```

muhammad-ahmad@latitude-7490:~/os/lab12$ ./Task5.out
Need Matrix:
1 0 0 0
0 0 1 0
0 2 0 0

```

Task 6:

Write the generic code for variable number of processes, resources and instances of each resource. Initialize the allocation matrix, maximum instance requirement matrix.

1. Enter the number of processes
2. Enter the number of resources
3. Initialize the resources with their maximum instances.
4. Allocate the resource instances to each process
5. Initialize the Maximum requirement of resource instances for each process
6. Print the maximum instances each resource has.
7. Print the allocation matrix
8. Print the maximum requirement matrix

Code:

```

#include <stdio.h>

int main() { int p, r;

/* 1. Enter number of processes */
printf("Enter number of processes: ");
scanf("%d", &p);

/* 2. Enter number of resources */
printf("Enter number of resources: ");
scanf("%d", &r);

int max_instances[r];
int allocation[p][r];
int max_required[p][r];

/* 3. Initialize maximum instances of each resource */
printf("\nEnter maximum instances of each resource:\n");

```

```

for (int i = 0; i < r; i++)
{
    printf("Resource R%d: ", i + 1);
    scanf("%d", &max_instances[i]);
}

/* 4. Allocate resource instances to each process */
printf("\nEnter Allocation Matrix:\n");
for (int i = 0; i < p; i++)
{
    for (int j = 0; j < r; j++)
    {
        printf("Allocation for P%d R%d: ", i + 1, j + 1);
        scanf("%d", &allocation[i][j]);
    }
}

/* 5. Initialize maximum requirement matrix */
printf("\nEnter Maximum Requirement Matrix:\n");
for (int i = 0; i < p; i++)
{
    for (int j = 0; j < r; j++)
    {
        do
        {
            printf("Max requirement for P%d R%d: ", i + 1, j + 1);
            scanf("%d", &max_required[i][j]);

            if (max_required[i][j] > max_instances[j])
                printf("Error: Cannot exceed max instances of resource R%d\n", j
1);

        } while (max_required[i][j] > max_instances[j]);
    }
}

/* 6. Print maximum instances */
printf("\nMaximum instances of each resource:\n");
for (int i = 0; i < r; i++)
    printf("R%d = %d\n", i + 1, max_instances[i]);

/* 7. Print allocation matrix */
printf("\nAllocation Matrix:\n");
for (int i = 0; i < p; i++)
{
    for (int j = 0; j < r; j++)
        printf("%d ", allocation[i][j]);
    printf("\n");
}

/* 8. Print maximum requirement matrix */
printf("\nMaximum Requirement Matrix:\n");
for (int i = 0; i < p; i++)
{
    for (int j = 0; j < r; j++)

```

```

        printf("%d ", max_required[i][j]);
    printf("\n");
}

return 0;

}

```

Output:

```

muhammad-ahmad@latitude-7490:~/os/lab12$ ./Task6.out
Enter number of processes: 3
Enter number of resources: 3
Enter maximum instances of each resource:
Resource R1: 3
Resource R2: 2
Resource R3: 3
Enter Allocation Matrix:
Allocation for P1 R1: 1
Allocation for P1 R2: 2
Allocation for P1 R3: 2
Allocation for P2 R1: 2
Allocation for P2 R2: 1
Allocation for P2 R3: 0
Allocation for P3 R1: 0
Allocation for P3 R2: 1
Allocation for P3 R3: 1
Enter Maximum Requirement Matrix:
Max requirement for P1 R1: 2
Max requirement for P1 R2: 1
Max requirement for P1 R3: 1
Max requirement for P2 R1: 1
Max requirement for P2 R2: 1
Max requirement for P2 R3: 3
Max requirement for P3 R1: 3
Max requirement for P3 R2: 2
Max requirement for P3 R3: 3
Maximum instances of each resource:
R1 = 3
R2 = 2
R3 = 3

Allocation Matrix:
1 2 2
2 1 0
0 1 1
Maximum Requirement Matrix:
2 1 1
1 1 3
3 2 3

```

Task 7:

Write the code to calculate the need matrix and Available resource instances vector.

Code:

```
#include <stdio.h>
#define P 3
#define R 4

int main()
{
    int allocation[P][R] = {
        {0, 1, 0, 0},
        {1, 0, 0, 0},
        {0, 0, 1, 0}
    };

    int max_required[P][R] = {
        {1, 1, 0, 0},
        {1, 0, 1, 0},
        {0, 2, 1, 0}
    };

    int max_instances[R] = {1, 2, 1, 3};
    int need[P][R];
    int available[R];

    for (int j = 0; j < R; j++)
    {
        int sum = 0;
        for (int i = 0; i < P; i++)
            sum += allocation[i][j];
        available[j] = max_instances[j] - sum;
    }

    for (int i = 0; i < P; i++)
        for (int j = 0; j < R; j++)
            need[i][j] = max_required[i][j] - allocation[i][j];

    printf("Need Matrix:\n");
    for (int i = 0; i < P; i++)
    {
        for (int j = 0; j < R; j++)
            printf("%d ", need[i][j]);
        printf("\n");
    }

    printf("\nAvailable Vector:\n");
    for (int i = 0; i < R; i++)
        printf("%d ", available[i]);

    return 0;
}
```

Output:

```
muhammad-ahmad@latitude-7490:~/os/lab12$ ./Task7.out
```

Need Matrix:

```
1 0 0 0
0 0 1 0
0 2 0 0
```

Available Vector:

```
0 1 0 3
```

Task 8:

Using Need matrix and available resource instance vector in the task 7. Identify and print the processes which will create deadlock if they started immediately.

Code:

```
#include <stdio.h>

#define P 3
#define R 4

int main()
{
    int need[P][R] = {
        {1, 0, 0, 0},
        {0, 0, 1, 0},
        {0, 2, 0, 0}
    };

    int available[R] = {0, 1, 0, 3};

    printf("Processes causing deadlock if started immediately:\n");

    for (int i = 0; i < P; i++)
    {
        int canRun = 1;
        for (int j = 0; j < R; j++)
        {
            if (need[i][j] > available[j])
            {
                canRun = 0;
                break;
            }
        }
        if (!canRun)
            printf("P%d\n", i + 1);
    }

    return 0;
}
```



```

        {
            for (int k = 0; k < R; k++)
                available[k] += allocation[i][k];

            safe[count++] = i;
            finish[i] = 1;
            found = 1;
        }
    }

    if (!found)
    {
        printf("System is NOT in safe state\n");
        return 0;
    }

    printf("Safe Sequence: ");
    for (int i = 0; i < P; i++)
        printf("P%d ", safe[i] + 1);

    return 0;
}

```

Output:

```

muhammad-ahmad@latitude-7490:~/os/lab12$ ./Task9.out
System is NOT in safe state

```

